



中山大学计算机学院

计算机网络实验报告

实验2——网络编程

✧（2025学年秋季学期）

	姓名	学号	评分（按百分制）	实验组编号：
组长：	王胜伟	23336228	100	3
组员1：	林宏宇	23320093	100	3
组员2：	王一澄	23336233	100	3
组员3：	宋信贤	23336207	100	3

✳ 任务一：基于UDP的端到端通信

一、实验题目

- 1 基本要求：基于套接字（Socket）编程实现端到端的UDP通信。
- 2 功能要求：发送端连续发送100个UDP数据包，接收端负责接收并统计最终丢失的数据包数量。
- 3 分析要求：在实验过程中，使用网络抓包工具Wireshark对通信数据包进行跟踪和分析。

二、实验代码及其讲解

1. UDP通信总体流程

本实验设计的UDP通信遵循以下流程：

- 1 系统初始化阶段：发送端和接收端都初始化Winsock库，创建各自的UDP套接字
- 2 服务端准备阶段：接收端创建UDP套接字，并将其绑定到指定的IP地址（172.16.3.3）和端口（8888）。为了避免无限等待，设置了5秒的接收超时时间，然后进入监听状态等待数据。
- 3 客户端连接阶段：发送端创建UDP套接字，并配置目标服务器的地址信息（IP: 172.16.3.3, Port: 8888）。
- 4 数据传输阶段：发送端通过一个循环，连续发送100个内容格式为 "Packet x" 的数据包。与此同时，接收端在循环中接收数据，并对收到的每个数据包进行解析和去重处理，以保证统计的准确性。
- 5 传输结束判断：发送端在发送完第100个包后自动结束。接收端如果在5秒内没有再收到任何数据，recvfrom 函数会因超时而返回，从而结束接收循环。
- 6 结果统计阶段：接收端根据记录的唯一数据包数量，计算出总的丢包数和丢包率，并将结果输出到控制台。
- 7 资源清理阶段：通信结束后，发送端和接收端均关闭各自的套接字，并调用WSACleanup() 来释放Winsock库所占用的资源。

2. 关键函数讲解

1. Winsock 初始化/清理 (`WSAStartup` / `WSACleanup`)

在Windows平台上进行Socket编程，必须首先初始化Winsock库。`WSAStartup` 函数加载网络功能模块，建立网络编程环境。程序结束时，必须调用 `WSACleanup` 来释放相关资源。

```
// 发送端和接收端都需要初始化
WSADATA wsaData;
if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
    std::cerr << "WSAStartup 失败" << std::endl;
    return 1;
}
// 程序结束前清理资源
WSACleanup(); // 清理Winsock资源，释放网络库
```

2. 创建套接字 (`socket`)

套接字是网络通信的端点。对于UDP通信，需要创建数据报套接字 (`SOCK_DGRAM`)。与TCP的流式套接字不同，UDP套接字是无连接的，每次发送数据都需要指定目标地址。

```
// 客户端创建套接字 (udp_sender.cpp第18行)
SOCKET clientSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
// 服务端创建套接字 (udp_receiver.cpp第21行)
SOCKET serverSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

// 参数说明:
// AF_INET: IPv4协议族，指定使用IPv4地址
// SOCK_DGRAM: 数据报套接字，UDP通信专用类型
// IPPROTO_UDP: 明确指定UDP协议，确保创建UDP套接字
```

3. 地址结构配置 (`sockaddr_in`)

`sockaddr_in` 结构体用于存储IPv4地址信息，包括协议族、端口号和IP地址。为了确保网络传输的正确性，端口号和IP地址需要转换为网络字节序。

```
// 服务端地址配置 (udp_receiver.cpp第25-28行)
sockaddr_in serverAddr;
serverAddr.sin_family = AF_INET; // 地址族设为IPv4
serverAddr.sin_port = htons(PORT); // 端口8888转换为网络字节序
serverAddr.sin_addr.s_addr = inet_addr("172.16.3.3"); // 服务端IP地址

// 客户端配置目标服务器地址 (udp_sender.cpp第25-28行)
sockaddr_in serverAddr;
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(SERVER_PORT); // 目标端口8888
serverAddr.sin_addr.s_addr = inet_addr(SERVER_IP); // 目标IP "172.16.3.3"
```

4. 绑定套接字 (**bind**) (仅服务端需要)

服务端需要将套接字绑定到指定的IP地址和端口号，这样客户端才能找到服务端。客户端通常不需要显式绑定，系统会自动分配可用端口。

```
// 服务端绑定操作 (udp_receiver.cpp第30-33行)
if (bind(serverSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
    SOCKET_ERROR) {
    std::cerr << "绑定失败, 错误代码: " << WSAGetLastError() << std::endl;
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
// 绑定成功后, 服务端就可以在172.16.3.3:8888上监听数据
```

5. 设置接收超时 (**setsockopt**)

为了避免接收端在没有数据时无限期地阻塞，我们设置了接收超时。当超过5秒未收到数据时，**recvfrom**将返回错误，程序可以据此判断数据传输已结束。

```
// 设置5秒接收超时 (udp_receiver.cpp第37-38行)
DWORD timeout = 5000; // 5秒超时, 单位为毫秒
setsockopt(serverSocket, SOL_SOCKET, SO_RCVTIMEO, (const char*)&timeout,
    sizeof(timeout));
// SOL_SOCKET: 套接字级别的选项
// SO_RCVTIMEO: 接收操作超时选项
```

6. 数据发送 (**sendto**) (客户端)

UDP是无连接协议，每次发送数据都需要指定目标地址。客户端使用**sendto**函数将数据包发送到指定的服务端地址，无需建立连接即可直接传输数据。

```
// 客户端循环发送100个数据包 (udp_sender.cpp第32-38行)
for (int i = 1; i ≤ 100; ++i) {
    char message[BUFFER_SIZE];
    sprintf(message, "Packet %d", i); // 格式化数据包内容

    sendto(clientSocket, message, strlen(message), 0, (SOCKADDR*)&serverAddr,
sizeof(serverAddr));
    std::cout << "已发送: " << message << std::endl;
}

// 参数详解:
// clientSocket: 发送方套接字
// message: 要发送的数据缓冲区, 包含"Packet 1"到"Packet 100"
// strlen(message): 实际数据长度
// 0: 发送标志, 通常为0
// serverAddr: 目标服务器地址结构 (172.16.3.3:8888)
```

7. 数据接收 (recvfrom) (服务端)

服务端使用recvfrom函数接收来自任意客户端的数据包。该函数会阻塞等待数据到达，同时返回发送方的地址信息，便于识别数据来源。

```
// 服务端循环接收数据包 (udp_receiver.cpp第45-55行)
while (true) {
    int bytesReceived = recvfrom(serverSocket, buffer, BUFFER_SIZE, 0,
(SOCKADDR*)&clientAddr, &clientAddrSize);

    if (bytesReceived == SOCKET_ERROR) {
        if (WSAGetLastError() == WSAETIMEDOUT) {
            std::cout << "\n[提示] 接收超时, 统计结束。" << std::endl;
            break;
        }
    }
    buffer[bytesReceived] = '\0'; // 添加字符串结束符
}

// 参数说明:
// serverSocket: 接收方套接字
// buffer: 接收数据的缓冲区 (1024字节)
// clientAddr: 返回发送方的地址信息 (IP和端口)
// 返回值: 实际接收的字节数, -1表示错误
```

8. 数据解析与去重

为了统计数据包的接收情况和计算丢包率，服务端需要解析每个数据包的序号，并使用集合容器进行去重处理，确保重复包不被重复统计。

```
// 数据包解析和统计 (udp_receiver.cpp第58-64行)
int packetNum = 0;
if (sscanf(buffer, "Packet %d", &packetNum) == 1) {
    if (receivedPackets.find(packetNum) == receivedPackets.end()) {
        packetCount++; // 增加唯一包计数
        receivedPackets.insert(packetNum); // 记录已接收的包序号
    }
}
// sscanf: 格式化字符串解析, 提取数据包序号
// std::set<int> receivedPackets: 利用set容器的唯一性特征去重
// 最终统计结果显示接收包数和丢包情况
```

9. 地址转换函数

网络编程中经常需要在二进制地址和可读字符串之间转换。inet_ntoa函数将网络字节序的IP地址转换为点分十进制格式，便于显示和调试。

```
// 显示发送方IP地址 (udp_receiver.cpp第67行)
char* clientIp = inet_ntoa(clientAddr.sin_addr);
std::cout << "收到来自 " << clientIp << " 的数据: " << buffer << std::endl;

// inet_ntoa: 将32位网络字节序IP地址转换为"192.168.1.1"格式
// 在本实验中显示客户端IP地址172.16.3.2
// 注意: inet_ntoa返回静态缓冲区指针, 不是线程安全的
```

10. 套接字关闭和资源清理

程序结束前必须正确关闭套接字并清理Winsock资源，避免资源泄漏。这是良好的编程习惯，确保系统资源得到正确释放。

```
// 发送端资源清理 (udp_sender.cpp第42-44行)
closesocket(clientSocket);
WSACleanup();
system("pause"); // 防止控制台窗口闪退

// 接收端资源清理 (udp_receiver.cpp第76-79行)
closesocket(serverSocket);
WSACleanup();
system("pause");

// closesocket: 关闭套接字, 释放套接字资源
// WSACleanup: 清理Winsock库, 与WSAStartup配对使用
// system("pause"): Windows特有, 等待用户按键后退出
```

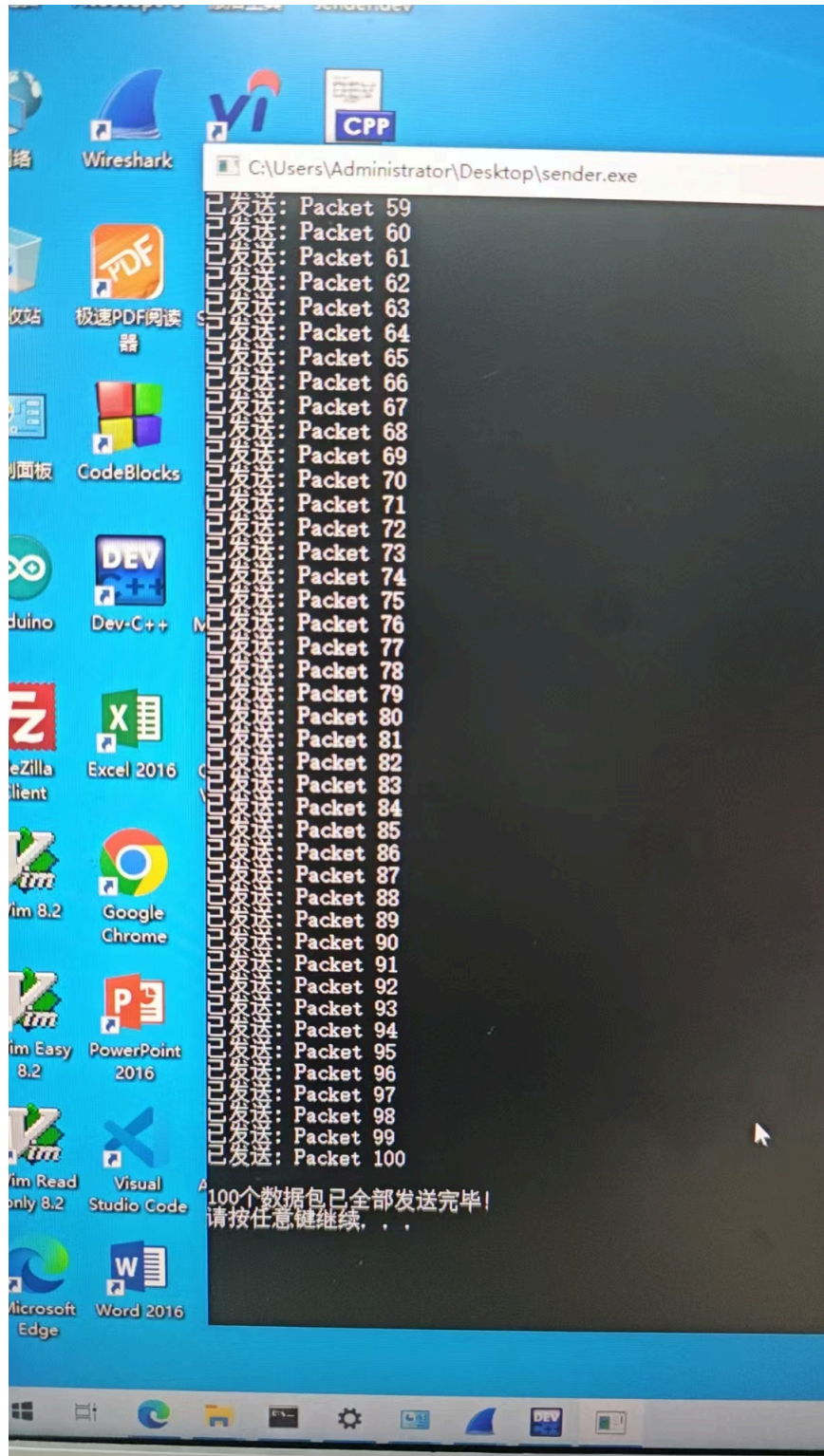
三、实验步骤

- ① 环境配置：配置两台主机，发送端IP为 **172.16.3.2**，接收端IP为 **172.16.3.3**，确保两台主机在同一局域网内且网络互通。
- ② 启动 **Wireshark**：在任意一台主机上打开Wireshark，选择正确的网络接口，并设置过滤器为 **udp**，开始抓包。
- ③ 运行接收端：编译并运行 **udp_receiver.cpp** 程序。程序将启动并显示“正在监听 172.16.3.3:8888...”，进入等待状态。
- ④ 运行发送端：编译并运行 **udp_sender.cpp** 程序。程序将立即开始连续发送100个UDP数据包。
- ⑤ 观察结果：
 - 在发送端控制台，可以看到从 "Packet 1" 到 "Packet 100" 的发送记录。
 - 在接收端控制台，可以看到实时收到的数据包内容和发送方地址。当数据发送完毕且超时后，接收端会打印出丢包统计结果。
- ⑥ 停止抓包与分析：在接收端程序结束后，停止Wireshark的抓包，并对捕获到的数据进行分析。

四、结果及分析

1. 程序运行结果

发送端输出：



接收端输出：


```
C:\Users\Administrator\Desktop\UdpReceiver.exe
程序已启动，本机IP: 172.16.3.3，正在端口 8888 等待数据...
收到来自 172.16.3.2 的数据: Packet 1
收到来自 172.16.3.2 的数据: Packet 2
收到来自 172.16.3.2 的数据: Packet 3
收到来自 172.16.3.2 的数据: Packet 4
收到来自 172.16.3.2 的数据: Packet 5
收到来自 172.16.3.2 的数据: Packet 6
收到来自 172.16.3.2 的数据: Packet 7
收到来自 172.16.3.2 的数据: Packet 8
收到来自 172.16.3.2 的数据: Packet 9
收到来自 172.16.3.2 的数据: Packet 10
收到来自 172.16.3.2 的数据: Packet 11
收到来自 172.16.3.2 的数据: Packet 12
收到来自 172.16.3.2 的数据: Packet 13
收到来自 172.16.3.2 的数据: Packet 14
收到来自 172.16.3.2 的数据: Packet 15
收到来自 172.16.3.2 的数据: Packet 16
收到来自 172.16.3.2 的数据: Packet 17
收到来自 172.16.3.2 的数据: Packet 18
收到来自 172.16.3.2 的数据: Packet 19
收到来自 172.16.3.2 的数据: Packet 20
收到来自 172.16.3.2 的数据: Packet 21
收到来自 172.16.3.2 的数据: Packet 22
收到来自 172.16.3.2 的数据: Packet 23
收到来自 172.16.3.2 的数据: Packet 24
收到来自 172.16.3.2 的数据: Packet 25
收到来自 172.16.3.2 的数据: Packet 26
收到来自 172.16.3.2 的数据: Packet 27
收到来自 172.16.3.2 的数据: Packet 28
收到来自 172.16.3.2 的数据: Packet 29
收到来自 172.16.3.2 的数据: Packet 30
收到来自 172.16.3.2 的数据: Packet 31
收到来自 172.16.3.2 的数据: Packet 32
收到来自 172.16.3.2 的数据: Packet 33
收到来自 172.16.3.2 的数据: Packet 34
收到来自 172.16.3.2 的数据: Packet 35
收到来自 172.16.3.2 的数据: Packet 36
收到来自 172.16.3.2 的数据: Packet 37
收到来自 172.16.3.2 的数据: Packet 38
收到来自 172.16.3.2 的数据: Packet 39
收到来自 172.16.3.2 的数据: Packet 40
收到来自 172.16.3.2 的数据: Packet 41
收到来自 172.16.3.2 的数据: Packet 42
收到来自 172.16.3.2 的数据: Packet 43
收到来自 172.16.3.2 的数据: Packet 44
收到来自 172.16.3.2 的数据: Packet 45
收到来自 172.16.3.2 的数据: Packet 46
收到来自 172.16.3.2 的数据: Packet 47
收到来自 172.16.3.2 的数据: Packet 48
收到来自 172.16.3.2 的数据: Packet 49
收到来自 172.16.3.2 的数据: Packet 50
收到来自 172.16.3.2 的数据: Packet 51
收到来自 172.16.3.2 的数据: Packet 52
收到来自 172.16.3.2 的数据: Packet 53
收到来自 172.16.3.2 的数据: Packet 54
收到来自 172.16.3.2 的数据: Packet 55
收到来自 172.16.3.2 的数据: Packet 56
收到来自 172.16.3.2 的数据: Packet 57
收到来自 172.16.3.2 的数据: Packet 58
收到来自 172.16.3.2 的数据: Packet 59
收到来自 172.16.3.2 的数据: Packet 60
收到来自 172.16.3.2 的数据: Packet 61
收到来自 172.16.3.2 的数据: Packet 62
```

```
C:\Users\Administrator\Desktop\UdpReceiver.exe
收到来自 172.16.3.2 的数据: Packet 48
收到来自 172.16.3.2 的数据: Packet 49
收到来自 172.16.3.2 的数据: Packet 50
收到来自 172.16.3.2 的数据: Packet 51
收到来自 172.16.3.2 的数据: Packet 52
收到来自 172.16.3.2 的数据: Packet 53
收到来自 172.16.3.2 的数据: Packet 54
收到来自 172.16.3.2 的数据: Packet 55
收到来自 172.16.3.2 的数据: Packet 56
收到来自 172.16.3.2 的数据: Packet 57
收到来自 172.16.3.2 的数据: Packet 58
收到来自 172.16.3.2 的数据: Packet 59
收到来自 172.16.3.2 的数据: Packet 60
收到来自 172.16.3.2 的数据: Packet 61
收到来自 172.16.3.2 的数据: Packet 62
收到来自 172.16.3.2 的数据: Packet 63
收到来自 172.16.3.2 的数据: Packet 64
收到来自 172.16.3.2 的数据: Packet 65
收到来自 172.16.3.2 的数据: Packet 66
收到来自 172.16.3.2 的数据: Packet 67
收到来自 172.16.3.2 的数据: Packet 68
收到来自 172.16.3.2 的数据: Packet 69
收到来自 172.16.3.2 的数据: Packet 70
收到来自 172.16.3.2 的数据: Packet 71
收到来自 172.16.3.2 的数据: Packet 72
收到来自 172.16.3.2 的数据: Packet 73
收到来自 172.16.3.2 的数据: Packet 74
收到来自 172.16.3.2 的数据: Packet 75
收到来自 172.16.3.2 的数据: Packet 76
收到来自 172.16.3.2 的数据: Packet 77
收到来自 172.16.3.2 的数据: Packet 78
收到来自 172.16.3.2 的数据: Packet 79
收到来自 172.16.3.2 的数据: Packet 80
收到来自 172.16.3.2 的数据: Packet 81
收到来自 172.16.3.2 的数据: Packet 82
收到来自 172.16.3.2 的数据: Packet 83
收到来自 172.16.3.2 的数据: Packet 84
收到来自 172.16.3.2 的数据: Packet 85
收到来自 172.16.3.2 的数据: Packet 86
收到来自 172.16.3.2 的数据: Packet 87
收到来自 172.16.3.2 的数据: Packet 88
收到来自 172.16.3.2 的数据: Packet 89
收到来自 172.16.3.2 的数据: Packet 90
收到来自 172.16.3.2 的数据: Packet 91
收到来自 172.16.3.2 的数据: Packet 92
收到来自 172.16.3.2 的数据: Packet 93
收到来自 172.16.3.2 的数据: Packet 94
收到来自 172.16.3.2 的数据: Packet 95
收到来自 172.16.3.2 的数据: Packet 96
收到来自 172.16.3.2 的数据: Packet 97
收到来自 172.16.3.2 的数据: Packet 98
收到来自 172.16.3.2 的数据: Packet 99
收到来自 172.16.3.2 的数据: Packet 100

【提示】接收超时，统计结束。

--- 统计结果 ---
总共收到了 100 个不重复的数据包。
理论上丢失了 0 个数据包。
请按任意键继续. . .
```

从接收端的最终统计结果“总共接收了 100 个不重复的数据包。丢包率为 0 / 100”可以看出，在本次局域网实验环境下，网络状况良好，100个UDP数据包全部被成功接收，没有发生丢包。

2. Wireshark抓包分析

[illegible]

The image displays two screenshots of the Wireshark network protocol analyzer. The top screenshot shows a list of captured packets, all of which are UDP packets from source IP 172.16.3.2 to destination IP 172.16.3.3. The bottom screenshot provides a detailed view of a selected packet (No. 1470), showing its structure: Ethernet II, Internet Protocol Version 4, and User Datagram Protocol (UDP). The UDP section indicates a source port of 61071 and a destination port of 8888. Below the protocol details, the raw packet data is shown in hexadecimal and ASCII format.

No.	Time	Source	Destination	Protocol	Length	Info
1470	24.708450	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1471	24.708654	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1472	24.708858	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1473	24.709084	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1474	24.709292	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1475	24.709498	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1476	24.709723	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1477	24.709930	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1478	24.710140	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1479	24.710355	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1480	24.710580	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1481	24.710791	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1482	24.710990	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1483	24.711216	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1484	24.711424	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1485	24.711624	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1486	24.711850	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=10

No.	Time	Source	Destination	Protocol	Length	Info
1453	24.704354	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1454	24.704561	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1455	24.704764	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1456	24.704991	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1457	24.705197	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1458	24.705399	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1459	24.705625	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1460	24.705829	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1462	24.706039	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1463	24.706264	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1464	24.706467	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1465	24.707362	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1466	24.707592	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1467	24.707822	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1468	24.708025	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1469	24.708226	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1470	24.708450	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1471	24.708654	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1472	24.708858	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1473	24.709084	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1474	24.709292	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1475	24.709498	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1476	24.709723	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1477	24.709930	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1478	24.710140	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1479	24.710355	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1480	24.710580	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1481	24.710791	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1482	24.710990	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1483	24.711216	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1484	24.711424	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1485	24.711624	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=9
1486	24.711850	172.16.3.2	172.16.3.3	UDP	60	61071 → 8888 Len=10

Frame 1470: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface \Device\NPF_{30DCE9B8-4053-45E5-9B55-D21CE09344D3}, id 0
 > Ethernet II, Src: HewlettP_3b:2a:9a (10:e7:c6:3b:2a:9a), Dst: HewlettP_3b:2a:5e (10:e7:c6:3b:2a:5e)
 > Internet Protocol Version 4, Src: 172.16.3.2, Dst: 172.16.3.3
 > User Datagram Protocol, Src Port: 61071, Dst Port: 8888
 > Data (9 bytes)

```

0000  10 e7 c6 3b 2a 5e 10 e7 c6 3b 2a 9a 08 00 45 00  ...;*.E-
0010  00 25 26 9a 00 00 00 11 06 04 ac 10 03 02 ac 10  -5a.....
0020  03 03 ee 8f 22 58 00 11 22 e5 50 61 63 60 65 74  -".....Packet
0030  20 38 34 00 00 00 00 00 00 00 00 00 00 00 00 00  84.....
  
```

上图是本次实验的Wireshark抓包截图，可以分析出以下几点：

- 协议与地址：所有捕获的数据包都使用了 **UDP** 协议。源IP地址（Source）为 **172.16.3.2**（发送端），目标IP地址（Destination）为 **172.16.3.3**（接收端），与代码配置一致。
- 端口信息：源端口（Source Port）是一个由操作系统动态分配的临时端口（如 **62541**），目标端口（Destination Port）是接收端绑定的固定端口 **8888**。
- 无连接特性：在数据传输开始之前，没有任何像TCP那样的“握手”建立连接的过程。发送端直接开始发送数据，体现了UDP无连接的特点。
- 数据包独立性：每个UDP数据包都是一个独立的单元，包含了完整的源/目标地址和端口信息。从截图中可以看到，每个包都被Wireshark独立解析。
- 尽力而为的传输：UDP不提供可靠性保证。虽然本次实验没有丢包，但在更复杂的网络环境中，这些数据包可能会丢失、重复或失序，而UDP协议本身不会进行任何重传或纠错。

✳ 任务二：基于TCP的回声服务器

一、实验题目

- ① 基本要求：基于TCP套接字（Socket）编程，实现一个简单的客户端/服务器（C/S）数据通信模型。
- ② 功能要求：客户端连接到服务器后，可以从控制台输入字符串并发送给服务器。服务器接收到字符串后，将其原样返回给客户端。
- ③ 分析要求：使用Wireshark对TCP连接建立、数据传输和连接断开的全过程进行跟踪与分析。

二、实验代码及其讲解

1. TCP通信总体流程

- ① 系统初始化阶段: 服务端和客户端都初始化Winsock库。
- ② 服务端准备阶段: 服务端创建TCP套接字（`SOCK_STREAM`），绑定到指定IP（`172.16.3.2`）和端口（`9999`），然后调用`listen`函数进入监听状态，等待客户端的连接请求。
- ③ 连接建立阶段: 客户端创建TCP套接字，并调用`connect`函数向服务器发起连接请求。服务器端通过`accept`函数接受该请求。此过程背后是TCP著名的“三次握手”，以建立一个可靠的连接。
- ④ 数据通信阶段: 连接建立后，客户端从用户处获取输入并使用`send`函数发送消息。服务端使用`recv`函数接收消息，然后立即使用`send`函数将收到的消息原样回传给客户端。客户端再通过`recv`接收并显示回声。这个过程可以循环进行。
- ⑤ 通信结束判断: 当客户端用户输入特定字符串（如"quit"）或直接关闭程序时，客户端会关闭连接。服务端检测到`recv`返回0时，判定客户端已断开连接。
- ⑥ 连接断开阶段: 任何一方关闭套接字都会启动TCP的“四次挥手”过程，以确保双方数据都已传输完毕，从而正常断开连接。
- ⑦ 资源清理阶段: 双方都关闭套接字并清理Winsock资源。

2. 关键函数讲解

1. Winsock 初始化/清理

TCP通信同样需要初始化Windows Socket库，这是所有Windows网络编程的基础步骤。初始化后才能使用各种Socket API函数进行网络通信。

```
// 服务端和客户端都需要初始化 (来自tcp_server.cpp和tcp_client.cpp)
WSADATA wsaData;
if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
    std::cerr << "WSAStartup 失败" << std::endl;
    return 1;
}

// 程序结束前清理资源
WSACleanup(); // 清理Winsock资源，释放网络库
```

2. 创建套接字，TCP使用AF_INET + SOCK_STREAM

TCP通信需要创建流式套接字，这是面向连接的可靠传输协议。与UDP的数据报套接字不同，TCP套接字提供可靠的、有序的数据传输。

```
// 服务端创建监听套接字 (tcp_server.cpp第17行)
SOCKET listenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
// 客户端创建连接套接字 (tcp_client.cpp第18行)
SOCKET clientSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

// 参数说明:
// AF_INET: IPv4协议族，指定使用IPv4地址
// SOCK_STREAM: 流式套接字，TCP通信专用类型，提供可靠连接
// IPPROTO_TCP: 明确指定TCP协议，确保创建TCP套接字
```

3. 地址结构配置

TCP通信中的地址配置与UDP类似，但更注重连接的准确性。服务端需要绑定到指定地址等待连接，客户端需要指定目标服务器地址。

```
// 服务端地址配置 (tcp_server.cpp第23-27行)
sockaddr_in serverAddr;
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT); // 端口9999转换为网络字节序
inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr); // 服务端IP "172.16.3.2"

// 客户端配置目标服务器地址 (tcp_client.cpp第25-28行)
sockaddr_in serverAddr;
serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(SERVER_PORT); // 目标端口9999
inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr); // 目标IP "172.16.3.2"
```

4. 套接字绑定（服务端专用）

TCP服务端必须将套接字绑定到指定的IP地址和端口号，这是建立监听服务的前提条件。绑定成功后，服务端就可以在该地址上接受客户端连接请求。

```
// 服务端绑定操作 (tcp_server.cpp第29-33行)
if (bind(listenSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
    SOCKET_ERROR) {
    std::cerr << "绑定失败, 错误代码: " << WSAGetLastError() << std::endl;
    closesocket(listenSocket);
    WSACleanup();
    return 1;
}
// 绑定成功后, 服务端就可以在172.16.3.2:9999上提供TCP服务
```

5. 监听连接请求（服务端专用）

TCP服务端需要调用listen函数开始监听客户端的连接请求。该函数将套接字设置为被动监听模式，准备接受传入的连接。

```
// 服务端开始监听 (tcp_server.cpp第35-39行)
if (listen(listenSocket, SOMAXCONN) == SOCKET_ERROR) {
    std::cerr << "监听失败, 错误代码: " << WSAGetLastError() << std::endl;
    closesocket(listenSocket);
    WSACleanup();
    return 1;
}
// SOMAXCONN: 系统允许的最大挂起连接数
// 监听成功后, 套接字进入被动等待状态
```


6. 接受客户端连接（服务端专用）

accept函数用于接受客户端的连接请求，完成TCP三次握手过程。该函数会阻塞等待，直到有客户端连接到达，然后返回一个新的套接字用于与该客户端通信。

```
// 服务端接受连接 (tcp_server.cpp第46-51行)
sockaddr_in clientAddr;
int clientAddrSize = sizeof(clientAddr);
SOCKET clientSocket = accept(listenSocket, (SOCKADDR*)&clientAddr,
&clientAddrSize);
if (clientSocket == INVALID_SOCKET) {
    std::cerr << "接受连接失败, 错误代码: " << WSAGetLastError() << std::endl;
    closesocket(listenSocket); WSACleanup(); return 1;
}
// accept返回新的套接字专门用于与该客户端通信
// 监听套接字listenSocket可以继续接受其他客户端连接
```

7. 连接服务器（客户端专用）

客户端使用connect函数主动发起与服务器的连接请求，启动TCP三次握手过程。连接成功后，客户端就可以与服务器进行双向数据通信。

```
// 客户端连接服务器 (tcp_client.cpp第30-38行)
if (connect(clientSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
    std::cerr << "连接服务器 " << SERVER_IP << " 失败, 错误代码: " <<
WSAGetLastError() << std::endl;
    system("pause");
    closesocket(clientSocket);
    WSACleanup();
    return 1;
}
std::cout << "[提示] 成功连接到服务器 " << SERVER_IP << ":" << SERVER_PORT <<
std::endl;
// connect成功表示TCP连接建立, 可以开始数据传输
```

8. TCP数据发送

TCP使用send函数发送数据，与UDP的sendto不同，TCP不需要指定目标地址，因为连接已经建立。send函数保证数据的可靠传输和正确顺序。

```
// 客户端发送数据 (tcp_client.cpp第54-59行)
int bytesSent = send(clientSocket, userInput.c_str(), userInput.length(), 0);
if (bytesSent == SOCKET_ERROR) {
    std::cerr << "[错误] send 失败, 错误代码: " << WSAGetLastError() << std::endl;
}
```

```

        break;
    }

    // 服务端回传数据 (tcp_server.cpp第65行)
    send(clientSocket, buffer, bytesReceived, 0);
    // 参数说明:
    // clientSocket: 已连接的套接字
    // buffer/userInput: 要发送的数据缓冲区
    // bytesReceived/length: 数据长度
    // 0: 发送标志, 通常为0

```

9. TCP数据接收

TCP使用recv函数接收数据，该函数会阻塞等待数据到达。与UDP的recvfrom不同，TCP不需要获取发送方地址，因为连接是点对点的。

```

// 服务端接收数据 (tcp_server.cpp第59-63行)
char buffer[BUFFER_SIZE];
int bytesReceived;
while ((bytesReceived = recv(clientSocket, buffer, BUFFER_SIZE, 0)) > 0) {
    buffer[bytesReceived] = '\0';
    std::cout << "收到消息: " << buffer << std::endl;
}

// 客户端接收回声 (tcp_client.cpp第61-70行)
int bytesReceived = recv(clientSocket, buffer, BUFFER_SIZE, 0);
if (bytesReceived > 0) {
    buffer[bytesReceived] = '\0';
    std::cout << "服务器回声: " << buffer << std::endl;
}

// 返回值: >0表示接收的字节数, 0表示连接关闭, -1表示错误

```

10. 地址转换函数（现代版本）

TCP代码中使用了更现代的地址转换函数inet_pton和inet_ntop，这些函数比传统的inet_addr和inet_ntoa更安全、功能更强。

```

// 设置服务器地址 (tcp_server.cpp第27行和tcp_client.cpp第28行)
inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr);

// 显示客户端IP地址 (tcp_server.cpp第53-54行)
char clientIp[INET_ADDRSTRLEN];
inet_ntop(AF_INET, &clientAddr.sin_addr, clientIp, INET_ADDRSTRLEN);
std::cout << "\n[提示] 客户端 " << clientIp << ":" << ntohs(clientAddr.sin_port)
<< " 已连接。" << std::endl;

// inet_pton: 将字符串IP地址转换为网络字节序二进制格式
// inet_ntop: 将网络字节序二进制地址转换为字符串格式
// 这些函数支持IPv4和IPv6, 比老版本函数更通用

```

11. 连接状态检测和错误处理

TCP通信中需要检测连接状态, 正确处理连接断开和各种错误情况, 确保程序的健壮性。

```

// 检测连接断开 (tcp_server.cpp第67-73行)
if (bytesReceived == 0) {
    std::cout << "[提示] 客户端已正常断开连接。" << std::endl;
}
else {
    std::cerr << "[错误] recv 失败, 错误代码: " << WSAGetLastError() << std::endl;
}

// 客户端检测服务器断开 (tcp_client.cpp第66-74行)
else if (bytesReceived == 0) {
    std::cout << "[提示] 服务器已关闭连接。" << std::endl;
    break;
}
// bytesReceived == 0: 对方正常关闭连接
// SOCKET_ERROR: 网络错误或异常断开

```

12. 套接字关闭和资源清理

TCP程序结束时需要关闭所有套接字并清理资源。服务端通常需要关闭监听套接字和客户端套接字两个套接字。

```
// 服务端资源清理 (tcp_server.cpp第75-78行)
closesocket(clientSocket); // 关闭与客户端的连接套接字
// 注意: 监听套接字在accept后已经关闭 (第56行)
WSACleanup();
system("pause");

// 客户端资源清理 (tcp_client.cpp第77-80行)
closesocket(clientSocket); // 关闭与服务器的连接套接字
WSACleanup();

// TCP连接是双向的, 任何一方都可以主动关闭连接
// 关闭套接字会触发TCP四次挥手过程, 正常断开连接
```

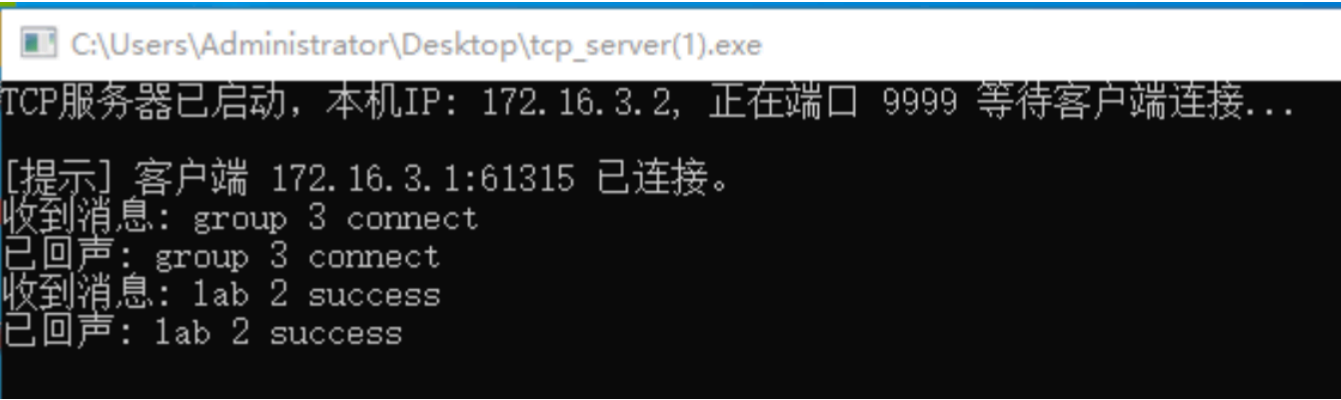
✧ 三、实验步骤

- ① 环境配置: 配置服务端IP为 172.16.3.2, 客户端IP为 172.16.3.1, 确保网络互通。
- ② 启动Wireshark: 在任一主机上启动Wireshark, 设置过滤器为 tcp.port == 9999, 开始抓包。
- ③ 运行服务端: 编译并运行 tcp_server.cpp, 服务端启动并监听在 9999 端口。
- ④ 运行客户端: 编译并运行 tcp_client.cpp, 客户端将尝试连接到服务端。
- ⑤ 交互通信:
 - 连接成功后, 在客户端输入任意消息 (如 "group 3 connect") 并回车。
 - 观察服务端是否收到消息并打印, 同时观察客户端是否收到了服务端返回的回声。
 - 可进行多次收发测试。
- ⑥ 结束通信: 在客户端输入 "quit" 或直接关闭窗口, 断开连接。
- ⑦ 分析抓包: 停止Wireshark抓包, 分析整个TCP会话的流程。

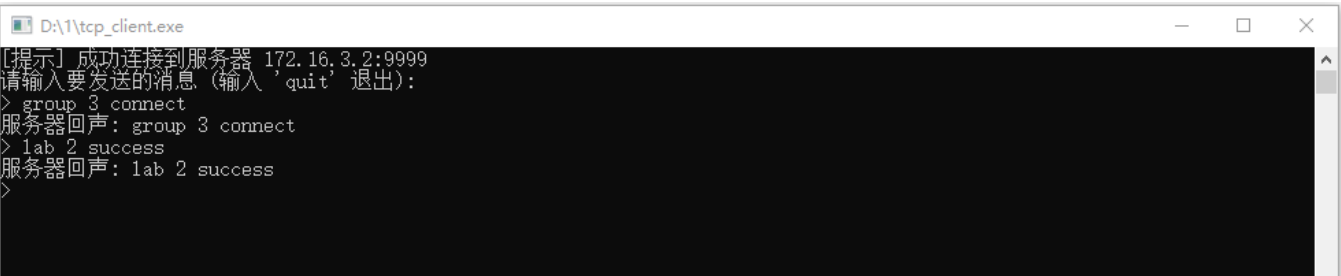
四、结果及分析

1. 程序运行结果

服务端输出：



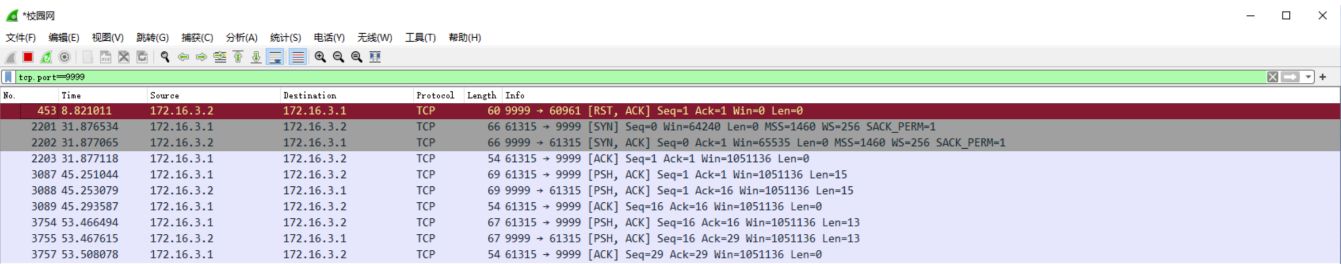
客户端输出：



结果说明：

- 1 服务端在 172.16.3.2 的 9999 端口成功启动监听。
- 2 客户端 172.16.3.1 成功连接到服务器。
- 3 客户端发送了 "group 3 connect"，服务端成功接收并回传，客户端也成功收到回声。
- 4 客户端随后发送了 "lab 2 success"，同样被服务端成功回声。
- 5 整个通信过程符合预期，功能正常。

2. Wireshark抓包分析



上图是本次TCP通信的Wireshark抓包截图，清晰地展示了TCP协议的关键阶段：

1. TCP三次握手（连接建立）

- 第一次握手 (SYN): 客户端 (172.16.3.1) 向服务端 (172.16.3.2) 发送一个 SYN 包

（同步序列编号），请求建立连接。Seq=0。

- 第二次握手 (SYN, ACK): 服务端收到请求后，回复一个 SYN, ACK 包。ACK 表示对客户端 SYN 的确认 (Ack=1，表示期望收到序列号为1的包)，SYN 表示服务端也发起一个同步请求 (Seq=0)。
- 第三次握手 (ACK): 客户端收到服务端的 SYN, ACK 后，发送一个 ACK 包作为确认 (Ack=1)。至此，连接建立完成。

2. 数据传输阶段

- 客户端 -> 服务端 (PSH, ACK): 客户端发送数据（如 "group 3 connect"，长度15字节）。PSH 标志提示接收方尽快将数据交给应用层。Seq=1, Ack=1。
- 服务端 -> 客户端 (PSH, ACK): 服务端收到数据后，将其原样回传（回声服务），数据长度同样为15字节。Seq=1, Ack=16 (因为收到了15字节数据，1+15=16)。
- 客户端 -> 服务端 (ACK): 客户端发送一个 ACK 包，确认收到了服务端的15字节回声数据。Seq=16, Ack=16。
- 后续 "lab 2 success" 的传输过程与此类似。

3. TCP四次挥手（连接断开）

- 抓包中显示了一个 [RST, ACK] 包，这通常表示连接被重置（强制关闭），而不是正常的四次挥手 (FIN -> ACK -> FIN -> ACK)。这可能是由于程序直接被关闭或发生异常导致的，而不是通过正常的关闭流程退出。尽管如此，它也有效地终止了连接。

分析总结：

抓包结果完美地印证了TCP协议的理论知识。从三次握手建立可靠连接，到使用序列号(Seq)和确认号(Ack)确保数据有序、不丢失的传输，再到最后断开连接，每一步都清晰可见。这与UDP的“发后不理”形成鲜明对比，充分体现了TCP的可靠性。

✧ 思考与总结

1. 说明在实验过程中遇到的问题 and 解决方法。

- 问题一：防火墙拦截
 - 描述：程序运行时，客户端无法连接到服务端，connect 函数超时失败。

- 解决方法：检查Windows防火墙设置，发现防火墙阻止了程序监听指定端口。通过关闭防火墙或为应用程序创建入站规则，允许其通过指定端口进行通信，问题得到解决。
- 问题二：IP地址或端口错误
 - 描述：客户端和服务端代码中的IP地址或端口号不匹配，导致连接失败。
 - 解决方法：仔细检查并统一了客户端和服务端代码中的IP地址和端口号宏定义，确保客户端连接的是服务端正确监听的地址和端口。
- 问题三：忘记调用 `WSAStartup`
 - 描述：程序一运行，调用 `socket` 函数就直接返回-1（`INVALID_SOCKET`），通过 `WSAGetLastError()` 检查发现错误码是 `WSANOTINITIALISED`。
 - 解决方法：意识到在Windows下进行任何Socket操作前都必须先初始化Winsock库。在程序开头添加 `WSAStartup` 函数的调用，并在程序结尾添加 `WSACleanup` 配对调用，问题解决。

2. 给出程序详细的流程图和对程序关键函数的详细说明。

- 流程图和关键函数说明已在 任务一 和 任务二 的“实验代码及其讲解”部分详细给出。

3. 使用Socket API开发通信程序中的客户端程序和服务器程序时，各需要哪些不同的函数？

- 服务端特有函数：
 - `bind()`：将套接字绑定到一个本地地址（IP和端口）。这是服务器接受连接的前提。
 - `listen()`：使套接字进入监听模式，等待客户端连接。
 - `accept()`：从监听队列中接受一个客户端连接，并返回一个新的套接字用于与该客户端通信。
- 客户端特有函数：
 - `connect()`：主动向服务器发起连接请求。
- 共有函数：
 - `WSAStartup()` / `WSACleanup()`：初始化和清理Winsock库。

- `socket()`: 创建套接字。
- `send()` / `recv()` (TCP) 或 `sendto()` / `recvfrom()` (UDP): 发送和接收数据。
- `closesocket()`: 关闭套接字。

4. 解释 `connect()`、`bind()` 等函数中 `struct sockaddr * addr` 参数各个部分的含义,并用具体的数据举例说明。

`struct sockaddr *addr` 是一个通用的套接字地址结构指针。它是一个“基类”指针,实际使用时需要强制转换为具体的协议族地址结构,最常见的是IPv4的 `struct sockaddr_in`。

`struct sockaddr_in` 结构体定义如下:

```
struct sockaddr_in {
    short          sin_family; // 地址族, 如 AF_INET (IPv4)
    unsigned short sin_port;   // 端口号
    struct in_addr sin_addr;   // IP地址
    char           sin_zero[8]; // 填充字节, 未使用
};

struct in_addr {
    unsigned long  s_addr;      // 32位的IPv4地址
};
```

参数含义及举例:

假设我们要将一个服务器绑定到 IP `192.168.1.10` 的 `8080` 端口:

```
sockaddr_in serverAddr;

// 1. sin_family: 地址族
serverAddr.sin_family = AF_INET;
// 含义: 指定使用IPv4协议。这是必须设置的。

// 2. sin_port: 端口号
serverAddr.sin_port = htons(8080);
// 含义: 指定端口号为8080。
// 举例: htons()函数将主机字节序的8080转换为网络字节序, 这是网络传输的标准。

// 3. sin_addr.s_addr: IP地址
serverAddr.sin_addr.s_addr = inet_addr("192.168.1.10");
// 含义: 指定IP地址为 "192.168.1.10"。
// 举例: inet_addr()函数将点分十进制的IP字符串转换为32位的网络字节序整数。
// 也可以绑定到任意可用IP: serverAddr.sin_addr.s_addr = INADDR_ANY;
```

```
// 4. sin_zero: 填充位
// 含义: 为了使 sockaddr_in 和 sockaddr 结构体大小相同而存在, 必须全部填充为0。
// 举例: 通常使用 memset(&serverAddr.sin_zero, 0, 8); 来清零。

// 在调用 bind() 时:
bind(serverSocket, (struct sockaddr *)&serverAddr, sizeof(serverAddr));
// 这里的 (struct sockaddr *)&serverAddr 就是将具体的IPv4地址结构指针强制转换为了通用的地址结构指针。
```

5. 说明面向连接的客户端和面向非连接的客户端在建立Socket时有什么区别。

- 面向连接 (TCP) 客户端:
 - **Socket**类型: 创建时使用 `SOCK_STREAM`。
 - 连接过程: 在数据传输前, 必须使用 `connect()` 函数与服务器建立一个显式的连接 (三次握手)。连接成功后, 该Socket就与唯一的服务器端点绑定, 后续的 `send` / `recv` 操作不再需要指定地址。
- 面向非连接 (UDP) 客户端:
 - **Socket**类型: 创建时使用 `SOCK_DGRAM`。
 - 连接过程: 不需要建立连接。 `connect()` 函数也可以用于UDP Socket, 但其作用仅仅是在内核中记录默认的目标地址, 这样后续可以使用 `send()` 代替 `sendto()`。但本质上UDP仍然是无连接的。通常情况下, UDP客户端不调用 `connect`, 而是在每次发送数据时通过 `sendto()` 函数直接指定目标地址。

6. 说明面向连接的客户端和面向非连接的客户端在收发数据时有什么区别。面向非连接的客户端又如何判断数据发送结束的?

- 收发数据区别:
 - 面向连接 (TCP): 使用 `send()` 和 `recv()` 函数。数据被视为一个连续的、可靠的字节流。操作系统保证数据按序、无差错、不重复地到达。
 - 面向非连接 (UDP): 使用 `sendto()` 和 `recvfrom()` 函数。数据以独立的数据包 (Datagram) 形式发送。每个数据包都需要指定目标地址。不保证到达、顺序和完整性。

- **UDP判断数据发送结束:**

UDP协议本身没有“结束”的概念。判断数据传输结束必须由应用层自己来设计。常见方法有:

- ① 特定结束包: 发送方发送一个特殊内容的数据包 (如一个内容为"FIN"的字符串), 接收方收到后即认为传输结束。
- ② 超时机制: 如本实验中接收端的实现, 设置一个接收超时时间 (`SO_RCVTIMEO`)。如果在指定时间内没有收到新的数据包, 就认为对方已经发送完毕。
- ③ 约定数据包数量: 发送方在传输开始前, 先告诉接收方总共要发送多少个数据包。接收方收到约定数量的包后即认为结束。

7. 比较面向连接的通信和无连接通信,它们各有什优缺点?适合在哪些场合下使用?

- 面向连接的通信 (**TCP**):

- 优点: 可靠、有序、数据无差错、有流量控制和拥塞控制。
- 缺点: 开销大 (需要三次握手四次挥手)、协议复杂、传输效率相对较低。
- 适用场合: 对数据完整性和可靠性要求极高的应用, 如文件传输 (**FTP**)、电子邮件 (**SMTP**)、网页浏览 (**HTTP**)。

- 无连接通信 (**UDP**):

- 优点: 速度快、开销小、实现简单、支持广播和多播。
- 缺点: 不可靠、无序、可能丢包。
- 适用场合: 对实时性要求高但能容忍少量丢包的应用, 如在线游戏、视频会议、语音通话 (**VoIP**)、DNS查询。

8. 实验过程中使用Socket时是工作在阻塞方式还是非阻塞方式?通过网络检索阐述这两种操作方式的不同。

实验过程中, Socket默认工作在阻塞方式下。

- 阻塞方式 (**Blocking I/O**):

- 描述: 当应用程序调用一个阻塞式I/O操作函数时 (如 `accept`, `connect`, `recv`,

`send`），如果该操作不能立即完成，则应用程序的执行流会被挂起（阻塞），直到操作完成或发生错误。

- 举例：调用 `recv()` 时，如果内核缓冲区没有数据，程序会一直停在 `recv()` 这一行，直到有数据到来。
- 优点：编程模型简单，逻辑清晰。
- 缺点：一个线程在同一时间只能处理一个Socket，如果需要同时处理多个连接，效率低下，通常需要为每个连接创建一个线程，造成资源浪费。

- 非阻塞方式 (Non-blocking I/O):

- 描述：当应用程序调用一个非阻塞式I/O操作函数时，该函数会立即返回，无论操作是否完成。如果操作不能立即完成，它会返回一个错误码（如 `EWOULDBLOCK`）。
- 举例：调用 `recv()` 时，如果内核缓冲区没有数据，函数不会等待，而是立即返回一个错误。程序可以继续执行其他任务，并需要通过轮询的方式反复检查Socket是否准备好。
- 优点：一个线程可以管理多个Socket，资源利用率高。
- 缺点：编程模型复杂，需要不断轮询检查状态，消耗CPU。通常与I/O多路复用技术（如 `select`，`poll`，`epoll`）结合使用，以获得高性能。

9. 引起UDP丢包的可能原因是什么？如何解决？

- 可能原因：

- ① 网络拥塞：当网络流量过大，超出路由器处理能力时，路由器会丢弃部分数据包。
- ② 接收端缓冲区满：接收端应用程序处理数据的速度跟不上数据到来的速度，导致操作系统内核的接收缓冲区被填满，后续到达的数据包会被丢弃。
- ③ 物理链路错误：网络线路质量差，或硬件设备故障，导致数据在传输过程中损坏，校验和（Checksum）验证失败而被丢弃。

- 解决方法：

由于UDP本身不提供可靠性，解决方案需要由应用层来实现：

- ① 实现确认和重传机制：接收方每收到一个数据包，就向发送方回复一个确认包（ACK）。发送方如果在一定时间内没有收到ACK，就重新发送该数据包。
- ② 序号机制：为每个数据包添加唯一的序列号。接收方可以根据序列号检测丢包和处理乱序，并请求重传丢失的包。

- ③ 流量控制/速率控制: 发送方根据网络状况动态调整发送速率, 避免过快发送导致网络拥塞或接收端缓冲区溢出。
- ④ 前向纠错 (**FEC**): 发送方在发送数据时加入冗余的纠错码, 接收方即使丢失了部分数据, 也能利用纠错码恢复出原始数据。
- ⑤ 切换到**TCP**: 如果应用的可靠性要求远大于实时性要求, 最简单的解决方案就是直接使用TCP协议。

✧ 参考资料

- ① Microsoft Docs. (2025). Winsock API Reference.
- ② Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Education.
- ③ Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley Professional.
- ④ 课堂PPT , CSDN文档等